

PDF Studio

User Manual

Version 2.8

A free, full-featured PDF reader and editor

Leon Priest · github.com/7h3v01d

Apache License 2.0 — free to use, modify, and share

Contents

1	About PDF Studio
2	Installing and running
3	The interface
4	Appearance and accessibility
5	Opening documents
6	Viewing and navigating
7	Searching
8	Annotations and markup
9	Signatures and stamps
10	Filling in forms
11	Redactions
12	Managing pages
13	Merging, splitting, extracting
14	Password protection
15	OCR — making scans searchable
16	Exporting to Word and Excel
17	Saving and printing
18	File associations (Windows)
19	Keyboard shortcuts
20	Building from source
21	Troubleshooting
22	Licence and credits

1. About PDF Studio

PDF Studio is a free, full-featured PDF reader and editor for Windows, built with Python, PyQt6, and PyMuPDF.

It is **free in every sense**: there is no trial, no activation, no licence key, and no feature is locked behind a paywall. The source is released under the Apache License 2.0.

Highlights

- View, annotate, mark up, and sign PDFs
- Fill in interactive PDF forms
- Insert, delete, reorder, rotate, merge, split, and extract pages
- Apply true redactions
- AES-256 password protection
- OCR scanned documents to make them searchable
- Open Word and Excel documents
- Export PDFs to Word and Excel
- Two accessibility-focused themes with an app-wide text-size control

2. Installing and running

Requirements

- Windows 10 or 11 (the app also runs on Linux/macOS from source)
- Python 3.10+ (*only if running from source*)

Option A — Run the built executable

Double-click `PDF_Studio.exe`. Nothing to install.

Option B — Run from source

```
cd PDF_Studio/src
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r ../requirements.txt
python pdf_reader.py
```

Optional features

Some features rely on additional components. The app works fine without them and will tell you what is missing if you try to use one.

Feature	Python packages	External programs
Open Word / Excel	(none)	Microsoft Office (best fidelity) or LibreOffice
Export to Word	pdf2docx	—
Export to Excel	tabula-py , openpyxl , pandas	Java
OCR	pytesseract , pdf2image , Pillow	Tesseract-OCR, Poppler

3. The interface

Area	What it does
Menu bar	File, Edit, View, Pages, Tools, Help
Main toolbar	Open, Save, Print, page navigation, zoom, rotate, full screen, search
Markup toolbar	Note, Highlight, Underline, Strikethrough, Draw, Eraser, colour, Signature, Stamp
Navigation panel (left)	Contents, Bookmarks, Annotations, Thumbnails
Page area (centre)	The document itself
Status bar (bottom)	Current page, zoom, rotation, view mode

The **Navigation panel** has four collapsible sections. Click a section header to expand or collapse it; drag the dividers to resize. Its layout is remembered between sessions. Toggle the whole panel with **F4**.

An asterisk (*) in the title bar means you have **unsaved changes**.

4. Appearance and accessibility

PDF Studio was built with low-vision readers in mind. All settings persist between sessions.

Themes

View  **Appearance**, then choose:

- **High-Contrast Light** — near-black text on white (*default*)
- **Dark Industrial** — light text on an obsidian background

Neither is universally "better" for low vision — it varies by individual and by condition. Try both.

Text size

View  **Appearance**  **Text Size: Medium, Large** (*default*), or **Extra Large**. This scales menus, toolbars, panels, dialogs, and form-field text across the whole application, and enlarges toolbar icons to match.

Typeface

All interface text is set in **Atkinson Hyperlegible**, a typeface designed by the Braille Institute to maximise character distinction for low-vision readers. It is bundled with the app — no installation needed.

Dark page background

View  **Dark Background** inverts the *document page* itself (separate from the app theme), which can reduce glare on dense white pages. You can combine a light interface with a dark page, or vice versa.

5. Opening documents

File  **Open...** (`Ctrl+O`), or click **Open** on the toolbar. The **Open** button's dropdown arrow lists your 10 most recent files.

Supported formats

Type	Extensions
PDF	.pdf
Word	.docx , .doc , .rtf , .odt
Excel	.xlsx , .xls , .ods , .csv

How Word and Excel files are opened

Office documents are **converted to PDF** for viewing and markup. The title bar shows the original filename with (*imported*) appended.

PDF Studio picks the highest-fidelity converter available:

- 1. Microsoft Word / Excel** via COM automation — this is Office performing its own *Save as PDF*, so the result is an exact reproduction. Used automatically when Office is installed. Requires `pywin32` .

- 2. LibreOffice** (headless) — free, very faithful, but not guaranteed pixel-identical to Word for complex layouts.

3. If neither is installed, the app explains what to install rather than producing a poor conversion.

Note: Imported documents are opened as *PDFs*. Saving produces a PDF, not a Word file. This is a view-and-markup path, not round-trip Word editing.

Password-protected PDFs

If a PDF is encrypted, you'll be prompted for the password on open.

6. Viewing and navigating

Action	How
Next / previous page	Next / Prev , or → / ←
Jump to page	Type the number in the toolbar page box, press Enter
First / last page	Ctrl+Home / Ctrl+End
Zoom in / out	Zoom + / Zoom - , Ctrl++ / Ctrl+- , or Ctrl+Wheel
Fit width / page	Fit W / Fit Pg (Ctrl+Shift+H / Ctrl+Shift+F)
Set an exact zoom	Type a percentage in the zoom box
Rotate 90°	Rotate (Ctrl+R)
Full screen	Full (F11)
Continuous scroll	View ☒ Continuous Scroll
Show/hide nav panel	F4

Thumbnails in the navigation panel jump to any page on click, and support drag-and-drop reordering (see §12).

Bookmarks: Ctrl+B or **Pages** ☒ **Add Bookmark** bookmarks the current page. Use **+ Add / - Remove** in the Bookmarks section.

Contents shows the PDF's own table of contents (outline), when it has one.

7. Searching

1. Click the **Search** box (Ctrl+F).

2. Type your text and press **Enter** .

3. Move between hits with **Next** / **Prev**, or **F3** / **Shift+F3** .

Matches are highlighted on the page. Search covers the whole document.

If a scanned document returns no results, it has no text layer — run **OCR** first (§15).

8. Annotations and markup

Select a tool from the markup toolbar, then use the mouse on the page. Press **Esc** to put the tool down.

Tool	Use
 Note	Click the page to place a sticky note; type its text
Highlight	Drag across text
Underline	Drag across text
Strikethrough	Drag across text
 Draw	Freehand ink
Eraser	Removes nearby markup
 (colour)	Sets the markup colour (remembered between sessions)

The **Annotations** section of the navigation panel lists every annotation in the document. Click one to jump to it, or delete it from there.

All markup supports **Undo/Redo** (**Ctrl+Z** / **Ctrl+Y**).

Markup becomes part of the document when you **Save**.

9. Signatures and stamps

These are **visual ink signatures** — an image stamped onto the page. They are *not* cryptographic/digital signatures (PDF certificate signing), and PDF Studio does not claim to provide those.

Adding a signature

Click  **Signature** on the markup toolbar. You have two modes:

Import image file (*recommended*)

1. Choose **Import image file**.
2. Click **Choose image...** and select a PNG/JPG of your signature.
3. Leave **Remove white background** ticked for scans or phone photos — it keys out the white paper so only the ink is placed.
4. Click **Add Signature**, then **click the page** where it should go.

Draw signature

1. Draw in the canvas with the mouse. Adjust **Thickness** and **Ink Colour**; **Clear** to start over.
2. Click **Add Signature**, then **click the page** where it should go.

Drag and drop

The fastest route: **drag an image file from Explorer straight onto the page**. It is placed at the drop point. If the image has no transparency, the white background is removed automatically.

Sizing

Signatures are placed at a sensible default width (~200pt), aspect-ratio preserved, capped at half the page width. Use **Ctrl+Z** to undo a misplacement.

Stamps

☒ **Stamp** inserts a text stamp (e.g. *APPROVED*, *DRAFT*) — you'll be prompted for the text, then click to place it.

Signatures and stamps are embedded permanently on **Save**.

10. Filling in forms

If a PDF contains interactive form fields, PDF Studio renders them as live controls over the page, outlined in light blue, pre-populated with any existing values.

Field type	Behaviour
Text (single-line)	Click and type
Text (multi-line)	Click and type
Checkbox	Click to tick / untick
Radio button	Click to select
Dropdown (combo)	Click and choose

- Values are written back into the PDF as you edit.
- Field text scales with the **Text Size** setting (§4).
- Fields reposition correctly when you zoom or resize.
- **Tools** ▢ **Reset Form Fields** clears the form.

Click **Save** to commit your entries.

Known limitations: list-box fields are not yet supported, and complex radio groups with custom export values may not map perfectly.

11. Redactions

1. Select the redaction tool and drag a box over the content to remove.
2. Repeat for each area.
3. **Tools** ▢ **Apply Redactions**.

Applying a redaction is permanent and irreversible. The underlying text and images are removed from the file, not merely covered. Always keep an unredacted original (**File** ▢ **Save a Copy...**) before applying.

12. Managing pages

From the **Pages** menu:

Action	Effect
Insert Blank Page	Adds a blank page
Delete Page	Removes the current page
Move Page Up / Down	Reorders the current page

You can also **drag and drop thumbnails** in the navigation panel to reorder pages.

All page operations are undoable (**Ctrl+Z**), including drag-reordering.

13. Merging, splitting, extracting

Tools ▢ **Merge / Split PDFs...**

- **Merge** — combine several PDFs into one. Add files, set their order, choose an output path.

- **Split** — break a PDF into separate files.

Tools ▢ **Extract Pages...** — pull a page range or selection into a new PDF, leaving the original untouched.

14. Password protection

Tools ▢ **Password Protect...**

- Set an **open password** (required to view the document).
- Set a **permissions password** (controls what may be done with it).
- Toggle permissions: printing, copying, annotating, form filling, and more.
- Encryption is **AES-256**.

Protection is applied when you save.

If you lose the password, the document cannot be recovered. There is no back door.

15. OCR — making scans searchable

A scanned page is just an image: you cannot search or copy from it. OCR adds an **invisible text layer** beneath the image, so the page looks identical but becomes searchable and selectable.

Tools ▢ **Run OCR...**

1. Choose the scope: **all pages**, **current page**, or a **custom range**.
2. Choose the **language** (from your installed Tesseract language packs).
3. Choose to **save as a new file** or **overwrite the original**.
4. Run. Processing happens in the background with a progress bar — the app

stays usable.

Requirements: `pytesseract` , `pdf2image` , `Pillow` , plus **Tesseract-OCR** and **Poppler** installed and on the system `PATH` .

16. Exporting to Word and Excel

File ▢ **Export As** ▢

Microsoft Word (.docx) — preserves layout, text, images, and columns via `pdf2docx` . You can select a page range.

Microsoft Excel (.xlsx) — extracts *tables* into styled worksheets (headers, alternating row shading, frozen panes) via `tabula-py` . Requires **Java**.

Both run in the background with a progress bar.

Export quality depends on the source. A clean, text-based PDF converts well; a scanned or heavily designed one may need cleanup. Excel export finds tables — it is not a general PDF-to-spreadsheet converter.

17. Saving and printing

Action	Shortcut	Notes
Save	Ctrl+S	Writes changes to the current file
Save As...	Ctrl+Shift+S	Saves to a new file
Save a Copy...	—	Writes a copy, keeps editing the original
Print...	Ctrl+P	Any system printer
Properties	—	View document metadata

Annotations, markup, signatures, stamps, and form entries are all embedded on save. An asterisk (*) in the title bar indicates unsaved changes.

18. File associations (Windows)

To make Windows open PDFs in PDF Studio:

File ▢ Set as Default PDF App...

This registers PDF Studio (per-user; **no administrator rights required**) and opens the Windows *Default apps* page, where you select **PDF Studio** for `.pdf`.

Alternatively: right-click a PDF ▢ **Open with ▢ Choose another app ▢ PDF Studio ▢** tick *Always use this app*.

Windows 10/11 deliberately prevent applications from silently making themselves the default handler (anti-hijacking). A one-time manual confirmation is therefore always required — by design, not a limitation of this app.

Command line:

```
"PDF Studio.exe" --register           :: .pdf + Word/Excel
"PDF Studio.exe" --register --pdf-only :: .pdf only
"PDF Studio.exe" --unregister         :: remove associations
```

Or use `register_pdf.bat` / `unregister_pdf.bat` .

19. Keyboard shortcuts

File

Shortcut	Action
Ctrl+O	Open
Ctrl+S	Save
Ctrl+Shift+S	Save As
Ctrl+P	Print
Ctrl+Q	Quit

Edit

Shortcut	Action
Ctrl+Z	Undo
Ctrl+Y / Ctrl+Shift+Z	Redo
Ctrl+C	Copy selected text
Ctrl+F	Find
F3 / Shift+F3	Find next / previous

Navigation

Shortcut	Action
← / →	Previous / next page
Ctrl+← / Ctrl+→	Previous / next page
Ctrl+Home / Ctrl+End	First / last page
Ctrl+B	Add bookmark

View

Shortcut	Action
Ctrl++ / Ctrl+-	Zoom in / out
Ctrl+Wheel	Zoom in / out
Ctrl+Shift+H	Fit width
Ctrl+Shift+F	Fit page
Ctrl+R	Rotate 90°
F11	Full screen
F4	Toggle navigation panel
Esc	Cancel the active tool

20. Building from source

Build in a **clean, isolated environment**. If unrelated packages are visible to PyInstaller (a second Qt binding such as PyQt5, other projects on your path, etc.) the build may pull them in or abort.

Easiest:

```
build_clean.bat
```

Creates a throwaway `.buildenv` , installs only what's needed, builds, and leaves the executable in `src\dist\` .

Manual:

```
python -m venv .venv
call .venv\Scripts\activate.bat
python -m pip install pyinstaller
python -m pip install -r requirements.txt
cd src
python -m PyInstaller "PDF Studio.spec"
```

Always use `python -m PyInstaller` , **never a bare** `pyinstaller` . The bare command runs whichever copy is first on your `PATH` — often a *global* one that builds against your global environment, even when a `venv` appears active.

Whatever optional packages are installed at build time are bundled into the executable. To include Word export, ensure `pdf2docx` is installed before building.

Renaming the app: `APP_NAME` , `APP_VERSION` , and `COMPANY_NAME` at the top of `src/about_dialog.py` are the single source of truth for the title bar, About box, and menus. The executable's name is set in `src/PDF Studio.spec` .

21. Troubleshooting

A Word/Excel document won't open Install **Microsoft Office** (best fidelity) or **LibreOffice** (free). Large documents take a few seconds to convert.

"Missing pdf2docx" when exporting to Word (built .exe) `pdf2docx` wasn't installed in the environment the executable was built from. Install it and rebuild. The error dialog now reports the underlying import failure, which names the specific missing module.

"bootstrap.ini is corrupt" when opening a Word file from the .exe Fixed in v2.8. External programs are now launched with a cleaned environment. If you see this, you are running an older build — rebuild from current source.

Scanned PDF can't be searched It has no text layer. Run **Tools** ▢ **Run OCR...** (§15).

OCR fails Confirm **Tesseract-OCR** and **Poppler** are installed and on your `PATH` .

Excel export fails Confirm **Java** is installed.

Build aborts: "multiple Qt bindings packages" PyQt5 is visible in your build environment. Build in a clean venv (`build_clean.bat`).

Text is too small / hard to read **View** ▢ **Appearance** ▢ **Text Size** ▢ **Extra Large**, and try both themes (§4).

The mouse behaves oddly on the page A markup tool is active. Press `Esc` .

22. Licence and credits

PDF Studio — Copyright © 2025 Leon Priest. Licensed under the **Apache License, Version 2.0**. See `LICENSE.txt` .

Third-party components

Component	Licence
PyMuPDF (fitz)	AGPL-3.0 / commercial (Artifex)
PyQt6	GPL-3.0 / commercial (Riverbank)
Atkinson Hyperlegible	SIL Open Font License 1.1 — © 2020 Braille Institute of America

Optional: pdf2docx , pytesseract , pdf2image , Pillow , tabula-py , openpyxl , pandas , pywin32 .

See NOTICE for full attributions.

Note on redistribution: PyMuPDF and PyQt6 are licensed under the AGPL and GPL respectively. If you distribute a built executable publicly, those terms apply to the distributed binary. Sharing it privately (for example, with family) is unaffected.